Session Calendar — 20 Weeks × 100 Live Sessions

Schedule source of truth. Follow the modules listed here in order. Classroom pacing follows this calendar.

Setting	Value
Live class	**1 hour 15 minutes**
Days	Mon–Fri (5 sessions / week)
Length	**20 weeks**
Total live sessions	**100**
Live hours	100 × 1.25h = **125 hours**
Homework	~45–90 min after most sessions

Compressed option: 16 weeks is *not* recommended for this syllabus at 75 minutes/day. Use only with Saturday labs or fewer React/JPA outcomes.

Rule: One live session = **one primary focus** (one concept block + one mini-lab). Do not schedule two full modules in one 75-minute class.

Engagement rule: Start from a lab ticket (`15_Lab_Assets/`, or `GIT/labs/`, `FRONTEND/labs/`, `FRONTEND/REACT/labs/`), then open module notes for Definition/Purpose as needed.

Biweekly Assessment: After every even week (Weeks 2, 4, … 20), complete the matching paper from `16_Assessments/` (BW01–BW10) as homework or Friday wrap-up. Graded papers live only under `16_Assessments/` — not inside modules.

Shared topics (repo root): `GIT/` · `MYSQL/` · `FRONTEND/` (includes `FRONTEND/REACT/`) — scheduled here, authored outside this folder.

Typical class shape (timing lives here — not repeated in every module):

Minutes	Block
5	Open lab kit
25	Concept demo / notes
35	Lab (starter/broken)
10	Exit + homework

How to read the table

Column	Meaning
S#	Global session number (01–100)
Focus	What you practice *in class*

Column	Meaning
Module file	Notes to open
Homework	Must finish before next session

Module file: click the link (or keep **same** = previous session's file).

Authoring status: Foundations + Core Java + OOP Parts 1–3 + Collections + Exception Handling + File Handling + Git/GitHub + MySQL bridge + JDBC CRUD + Maven/Spring Boot setup (`01`) + Spring Web REST (`02`) + Spring Data JPA (`03`) + REST DTOs / Postman (`04`) + Spring Security lite (`05`), lab kits 01–19 (kit 19: JWT register/login + protect POST; Lab 06 freeze; Lab 07 Actuator/health + env/PaaS/M0), Servlet vs Spring interview map (`08`, kit 19 point-only), and BW01–BW06 exist. Shared FRONTEND HTML/CSS/JS is already used. Protected labs `06` (CRUD + auth freeze) and `07` (deploy debug, kit 19) are authored. Later phases (CORS recap via `04`, React, testing, capstone M1–M3) are still planned names until authored.

Phase A — Foundations & Core Java (Sessions 01–30)

S#	Wk	Focus	Module file	Homework
01	1	Computer, OS, files, terminal basics	01_Foundations/01_Computer_and_Programming_Foundations.md	Workspace folders
02	1	Client–server + roles; VS Code / IntelliJ tour	same	Roadmap note
03	1	Install JDK 17; verify PATH, `javac` / `java`	02_Core_Java/01_Java_Setup_and_First_Programs.md	Setup proof screenshot
04	1	`public class` + `main`; first programs	same	`HelloJava` on GitHub account created
05	1	Keywords, identifiers; lab catch-up	same	Flash keywords
06	2	Variables & primitive types	02_Core_Java/02_Variables_Datatypes_Operators_IO.md	Type drills
07	2	Operators	same	Calculator v1
08	2	`Scanner` I/O & printing	same	Canteen calculator + receipt print
09	2	Casting & wrappers lite	same	Fix 5 casting mistakes
10	2	Mini: CLI calculator polish	same	Push repo
11	3	`if` / `else`	02_Core_Java/03_Flow_Control.md	Grade calculator
12	3	`switch`	same	Menu with switch
13	3	`for` / `while`, break/continue	same	Tables & number guessing
14	3	Nested menus	same	Start ATM menu

S#	Wk	Focus	Module file	Homework
15	3	ATM menu lab day	same	Finish ATM + push
16	4	Strings: immutability, index, `substring`	02_Core_Java/04_Strings.md	Slice drills
17	4	String methods; `equals` vs `==`	same	Email/phone parse
18	4	Arrays 1D + enhanced `for`	02_Core_Java/05_Arrays.md	Marks array
19	4	2D arrays + simple algos	same	Max / search lab
20	4	Marks Manager mini-project	same	Push CLI app
21	5	Static methods, return, params	02_Core_Java/06_Methods.md	Extract calculator methods
22	5	Method overloading	same	Overload drills
23	5	Refactor ATM into methods (one class)	same	Methods-only ATM
24	5	Class / object / `new`	03_OOP_Java/03_OOP_Part1_Classes_Encapsulation.md	Student class
25	5	Constructors + `this`	same	Constructor drills
26	6	Instance methods vs static	same	Move logic onto objects
27	6	Encapsulation (`private`, getters)	same	Hide balance / marks
28	6	Instance vs static fields	same	Counter vs per-object data
29	6	BankAccount lab	same	Deposit / withdraw with `print` + `return`
30	6	Part 1 catch-up / push	same	Push OOP Part 1

Phase B — OOP, Collections, IO & Git (Sessions 31–45)

S#	Wk	Focus	Module file	Homework
31	7	Inheritance + `super()`	03_OOP_Java/03_OOP_Part2_Inheritance.md	Employee hierarchy start
32	7	Method overriding	same	Override `display` / `toString` lite
33	7	IS-A; single class inheritance	same	Sketch campus roles
34	7	Abstract classes	03_OOP_Java/03_OOP_Part3_Polymorphism_Interfaces.md	Abstract `Account`
35	7	Interfaces (`implements`)	same	`Payable` / `Printable`

S#	Wk	Focus	Module file	Homework
36	8	Polymorphism + packages; OOP mini-project push	same	Push Campus Wallet
37	8	`ArrayList` / `List`	04_Collections_Exceptions_IO/01_Collections.md	List CRUD
38	8	`HashSet` / uniqueness	same	Dedup emails
39	8	`HashMap` / lookups	same	Contact map
40	8	Collections mini-project	same	Push inventory / contacts
41	9	`try` / `catch` / `finally`; try-with-resources	04_Collections_Exceptions_IO/02_Exception_Handling.md	Safe `Scanner` lab
42	9	`throw` + custom exceptions	same	Domain errors
43	9	Files (`java.nio` / streams) + CSV/text	04_Collections_Exceptions_IO/03_File_Handling.md	CSV append
44	9	Git Session 1: first GitHub push	../GIT/01_Git_Session1.md	KSR_MART on GitHub
45	9	Git Session 2: daily loop + clone	../GIT/02_Git_Session2.md	Hygiene lab; homework Session 3 · 4 · 5

Phase C — MySQL & JDBC (Sessions 46–58)

External dependency: repo `MYSQL/` (and `FRONTEND/` / `FRONTEND/REACT/` on later weeks).

S#	Wk	Focus	Module / lab	Homework
46	10	Why DB; install/connect	MYSQL README + 00_MySQL_Setup + FS bridge	`training_db` created
47	10	Tables, PK; DDL CREATE	MYSQL/01–02	3 tables
48	10	DML INSERT/UPDATE/DELETE	MYSQL/02	Seed data
49	10	DQL WHERE/ORDER/LIMIT	MYSQL/03 + labs seed	Report queries
50	10	FS bridge concepts + ER sketch	06_Database_MySQL/01_MySQL_for_Full_Stack_Developers.md	Library ER
51	11	Modeling PK/FK	MYSQL/06	Related tables
52	11	INNER JOIN	MYSQL/07	Join drills
53	11	LEFT JOIN + multi-table	MYSQL/07	Customers w/o orders

S#	Wk	Focus	Module / lab	Homework
54	11	TCL + indexes lite	MYSQL/09	Rollback demo
55	11	JDBC: `Connection`, try-with-resources	07_JDBC/01_JDBC_CRUD.md	Connect proof (no Spring)
56	12	`PreparedStatement` SELECT	same	Parameterized SELECT
57	12	JDBC INSERT/UPDATE/DELETE	same	No string-concat SQL
58	12	JDBC Library CRUD mini + injection note	same	Push JDBC CRUD

Phase D — Frontend (Sessions 59–65)

S#	Wk	Focus	Module	Homework
59	12	HTML structure & semantic	../FRONTEND/01_HTML.md	Profile page
60	12	Forms & a11y basics	same	Labeled form
61	13	CSS box model + Flex	../FRONTEND/02_CSS_and_Responsive.md	Layout
62	13	Responsive + media queries	same	Mobile screenshot
63	13	JS variables, DOM, events	../FRONTEND/03_JavaScript_DOM_Forms.md	Click counter
64	13	Form validation	same	Validate contact
65	13	`fetch` JSON overview	same	Portfolio page push

Phase E — Spring Boot (Sessions 66–80)

S#	Wk	Focus	Module	Homework
66	14	Maven + Spring Boot init	09_Spring_Boot/01_Maven_and_Spring_Boot_Setup.md	`mvn spring-boot:run` proof
67	14	`application.yml` + first `@RestController`	same	Hello JSON
68	14	Mappings, JSON, status codes	09_Spring_Boot/02_Spring_Web_REST.md	GET list (in-memory)
69	14	`@PathVariable` / `@RequestBody`	same	POST + GET by id
70	14	Thymeleaf **overview only** (one page)	same	Optional HTML view note

S#	Wk	Focus	Module	Homework
71	15	JPA entity + repository	09_Spring_Boot/03_Spring_Data_JPA_Validation.md	Product entity
72	15	JPA CRUD + MySQL	same	Persist catalog
73	15	Validation + DTO mapping	09_Spring_Boot/04_REST_DTOs_Postman.md	`@Valid` on POST
74	15	Postman collection + error bodies	same	Collection v1
75	15	Security lite: register/login (JWT)	09_Spring_Boot/05_Spring_Security_Lite.md	Auth works
76	16	**Lab 06** CRUD/auth consolidation	09_Spring_Boot/06_Lab_CRUD_Auth_Consolidation.md	Feature freeze list
77	16	**Lab 06** continue + peer break-auth	same	Must-pass items
78	16	**Lab 06** demos	same	Deliverables bundle
79	16	**Lab 07** deploy debug	09_Spring_Boot/07_Lab_Deploy_Capstone_Debug.md	Actuator / health + env
80	16	**Lab 07** PaaS attempt + capstone M0	Capstone_CampusCart_One_Pager.md	Auth shell push

Phase F — Interview map, React, Quality, Capstone (Sessions 81–100)

S#	Wk	Focus	Module	Homework
81	17	Servlet vs Spring (interview map only)	09_Spring_Boot/08_Servlet_vs_Spring_Interview_Map.md	5 comparison bullets
82	17	CORS + API recap for React	09_Spring_Boot/04_REST_DTOs_Postman.md	CORS config note
83	17	Capstone M1 auth shell	Capstone one-pager	Login/register API
84	17	Capstone M2 catalog JPA	same	Product CRUD
85	17	Capstone M3 cart/orders start	same	Order flow stub
86	18	React Vite + first component	../FRONTEND/REACT/01_React_Fundamentals.md	Hello card
87	18	Props + state	same	Counter + list
88	18	Forms + lists/keys	same	Controlled form
89	18	`useEffect` + Router	../FRONTEND/REACT/02_React_Hooks_Routing_API_Deploy.md	Two routes

S#	Wk	Focus	Module	Homework
90	18	Fetch Spring API + CORS	same	Product list UI
91	19	Debugging + logging habits	12_Testing_Clean_Code/01_Testing_Debugging_Clean_Code.md	Debugger lab
92	19	JUnit 5 + Mockito lite	same	3 tests
93	19	Capstone M2–M3 features	Capstone one-pager	Orders flow
94	19	Capstone polish + a11y pass	Deliverables + one-pager	Screenshots
95	19	Deploy capstone / Lab 07 catch-up	Capstone one-pager	Live or staging URL
96	20	Resume + LinkedIn + GitHub pins	13_Placement/01_Career_Resume_Interview.md	Resume v1
97	20	Mock interview (Java + SQL)	same	Feedback notes
98	20	Mock interview (Spring + projects)	same	STAR stories
99	20	Capstone defense rehearsals	Capstone	10-min script
100	20	Demo day + placement wrap	Placement	Applications started

AI Integration (12_AI_Integration): optional after S95 — never displaces S93–S100.

If the batch falls behind

Slip React / JWT depth / Thymeleaf, not Spring Lab 06/07, JDBC `PreparedStatement`, or SQL joins. Those are placement-critical.